

Guía de Despliegue · AgroCredito

Objetivo: desplegar la aplicación en infraestructura propia del cliente (cuentas de GitHub, Google Earth Engine, Supabase y Streamlit Community Cloud), partiendo del archivo **agrocredito.zip** entregado por el equipo.

Tiempo estimado: 45–90 min · **Coste:** 0 € (todos los servicios disponen de plan gratuito).

Resumen del procedimiento

#	Paso	Servicio	Resultado
0	Crear las cuentas gratuitas	GitHub · Google Cloud · Supabase · Streamlit	Acceso a los servicios
1	Descomprimir agrocredito.zip	Equipo local	Carpeta con el código
2	Publicar el código en un repositorio propio	GitHub	Repositorio agrocredito
3	Configurar Google Earth Engine	Google Cloud	JSON de cuenta de servicio
4	Configurar la base de datos (opcional)	Supabase	Cadena DATABASE_URL
5	Desplegar la aplicación	Streamlit Community Cloud	URL de la aplicación
6	Cargar las credenciales (<i>Secrets</i>)	Streamlit Cloud -> Settings	Credenciales activas
7	Verificar el funcionamiento	Navegador	Aplicación operativa

Prueba rápida (sin base de datos): es posible desplegar y probar la aplicación en **modo de demostración** (2 predios de ejemplo) omitiendo el Paso 4. Únicamente se requiere Earth Engine (Paso 3). Para operar con predios reales, complete el Paso 4.

Credenciales necesarias (se cargan en el Paso 6):

- **DATABASE_URL** — cadena de conexión de Supabase (*solo con base de datos real*).
- **[gee] project** y **[gee] service_account_json** — de Google Earth Engine (*obligatorio*).

Procedimiento detallado

Paso 0 · Creación de las cuentas

Cree una cuenta en cada servicio (si aún no dispone de ellas), preferiblemente con el mismo correo corporativo:

1. **GitHub** — <https://github.com/signup>
2. **Google Cloud** (incluye Earth Engine) — <https://console.cloud.google.com>
 - Regístrese además en Earth Engine: <https://code.earthengine.google.com>

(acepte los términos y seleccione el tipo de uso que corresponda).

3. **Supabase** — <https://supabase.com> (*opcional, solo para predios reales*).
4. **Streamlit Community Cloud** — <https://share.streamlit.io>

(inicie sesión **con la cuenta de GitHub**).

Paso 1 · Descompresión del archivo

1. Descomprima **agrocredito.zip**. Se obtendrá una carpeta con **app.py**, **requirements.txt**, **utils/**, **datos/**, entre otros.
 2. No es necesario instalar nada en el equipo local: el despliegue se realiza en la nube.
-

Paso 2 · Publicación del código en GitHub

1. En GitHub, seleccione «**New repository**»:
 - Nombre: **agrocredito** · Visibilidad: **Private** (recomendado).
 - No marque «Add a README» (ya se incluye en el paquete).
2. Cargue los archivos. La vía más sencilla (sin línea de comandos):
 - En el repositorio vacío, use el enlace «**uploading an existing file**», arrastre **todo el contenido** de la carpeta descomprimida y confirme con «**Commit changes**».
 - (*Alternativa por línea de comandos, si se utiliza git:*)

```
cd carpeta_descomprimida
git init && git add . && git commit -m "AgroCredito"
git branch -M main
git remote add origin https://github.com/SU_USUARIO/agrocredito.git
git push -u origin main
```

3. **Importante** — **qué NO se debe subir a GitHub**:
 - El archivo **.streamlit/secrets.toml** (credenciales); se cargan solo en Streamlit (Paso 6).
 - La carpeta **datos/db/** (contiene la base de datos, de gran tamaño; solo se utiliza una vez para la restauración en Supabase del Paso 4 / Anexo A, y supera el límite de 100 MB de GitHub).
- El **.gitignore** incluido ya excluye ambos automáticamente si utiliza git. Si sube los archivos manualmente por la web, **no arrastre la carpeta datos/db/** ni **secrets.toml**.
-

Paso 3 · Configuración de Google Earth Engine (obligatorio)

Permite descargar el NDVI de Sentinel-2. Se requiere un **proyecto de Google Cloud** con la Earth Engine API habilitada y una **cuenta de servicio**.

1. En **Google Cloud Console**, cree un proyecto (anote su **ID**, p. ej. **agrocredito-cliente**).
2. En **APIs y servicios**, habilite «**Google Earth Engine API**».
3. En **IAM y administración** -> **Cuentas de servicio** -> **Crear cuenta de servicio**:
 - Nombre: **earth-engine** · Rol: *Viewer* (o *Earth Engine Resource Viewer*).

4. En la cuenta creada, abra **Claves -> Añadir clave -> Crear clave -> JSON**.

Se descargará un archivo **.json**: **consérvelo; se utilizará en el Paso 6**.

5. Autorice la cuenta de servicio en Earth Engine (una sola vez). Consulte:

https://developers.google.com/earth-engine/guides/service_account

Conserve el **ID del proyecto** y el **contenido del archivo .json**.

Paso 4 · Configuración de la base de datos Supabase (opcional)

Necesaria únicamente para consultar **predios reales** del catastro. Si se omite, la aplicación funciona en **modo de demostración** (ver más abajo).

1. En **Supabase**, cree un proyecto («**New project**»):

- **Importante — región:** seleccione una región de **EE. UU.** (p. ej.

East US · North Virginia). Streamlit Community Cloud se ejecuta en EE. UU., por lo que una base de datos en EE. UU. minimiza la latencia de cada consulta. Deben evitarse regiones lejanas (p. ej. São Paulo), que ralentizan notablemente la aplicación.

- Defina y **anote** la contraseña de la base de datos.

2. En **Database -> Extensions**, habilite la extensión **postgis**.

3. **Cargue la base de datos incluida en el paquete**. El zip contiene el archivo **datos/db/agrapp_postgis.dump** con las tablas ya preparadas. Siga el

Anexo A (al final de esta guía) para restaurarlo en Supabase.

4. Copie la **cadena de conexión** en ****Project Settings -> Database ->**

Connection string -> URI, con la forma

postgresql://postgres:SU_PASSWORD@db.SU_PROYECTO.supabase.co:5432/postgres. Esta cadena es la **DATABASE_URL**** del Paso 6.

Paso 5 · Despliegue en Streamlit Community Cloud

1. Acceda a <https://share.streamlit.io> (con la cuenta de GitHub).

2. Seleccione «**Create app -> Deploy a public app from GitHub**».

3. Configure:

- **Repository:** **SU_USUARIO/agrocredito**

- **Branch:** **main**

- **Main file path:** **app.py**

4. Pulse «**Deploy**». La primera construcción instala las dependencias de **requirements.txt** (varios minutos). Es normal que muestre un error de credenciales hasta completar el Paso 6.

Paso 6 · Carga de las credenciales (Secrets)

1. En la aplicación desplegada, abra el menú **... -> Settings -> Secrets**.

2. Pegue el siguiente contenido y complételo con sus datos (plantilla de referencia: **.streamlit/secrets.toml.example**):

```
# Solo si se utiliza base de datos real (Paso 4):
DATABASE_URL = "postgresql://postgres:SU_PASSWORD@db.SU_PROYECTO.supabase.co:5432/postgres"

[gee]
project = "su-proyecto-gcp"
service_account_json = ""
{ ...contenido COMPLETO del .json de la cuenta de servicio... }
""
```

3. Guarde con «**Save**». La aplicación se reinicia con las credenciales activas.

Paso 7 · Verificación

1. Abra la URL de la aplicación.
2. En la pestaña «Inicio», defina un predio (por punto, dibujo o GeoJSON) y pulse «Analizar predio».
3. En «Validación Pre-Crédito», se calcularán terreno, NDVI y actividad productiva (la primera ejecución tarda 1–2 min por las consultas a GEE).
4. En «Monitoreo & Forecast», descargue la plantilla, cárguela y pulse «Calcular indicadores».

Si se muestran resultados y mapas, el despliegue se ha completado correctamente.

Modo de demostración (sin base de datos)

Para probar la aplicación sin Supabase:

1. Edite `utils/postgis_client.py` y modifique
`USE_REAL_DB = True -> USE_REAL_DB = False.`
2. Suba el cambio a GitHub. La aplicación empleará 2 predios de ejemplo (Salento y Turbo) y no requerirá `DATABASE_URL`; únicamente Earth Engine (Paso 3).

Resolución de problemas frecuentes

Síntoma	Causa probable	Solución
«No se encontraron credenciales GEE»	Falta o formato incorrecto de <code>[gee]</code> en <code>Secrets</code>	Revise el Paso 6; el JSON debe ir completo entre <code>""</code>
Error al consultar el predio	<code>DATABASE_URL</code> ausente o base sin datos	Complete el Paso 4, o utilice el modo de demostración
La aplicación no arranca tras el despliegue	Falta alguna dependencia	Revise los <code>logs</code> en «Manage app»
NDVI vacío o sin escenas	Nubosidad alta o zona sin cobertura	Pruebe otras coordenadas o amplíe el periodo

Consideraciones de seguridad

- **Credenciales:** residen exclusivamente en los `Secrets` de Streamlit, nunca en el código ni en GitHub (el `.gitignore` incluido protege `secrets.toml`).
- **Repositorio privado:** Streamlit Community Cloud permite desplegar desde repositorios **privados** de GitHub; se recomienda mantener el repositorio en modo privado.
- **Importante — visibilidad de la aplicación desplegada:** en el plan gratuito de Streamlit Community Cloud, la aplicación es **accesible públicamente a través de su URL**, aunque el repositorio de código sea privado. Para reducir el riesgo de exposición de datos se recomienda:
 - Verificar en «Manage app -> Settings -> Sharing» si la cuenta permite **restringir la audiencia** a correos autorizados y, de estar disponible, activarlo.
 - No exponer en la interfaz datos sensibles o identificables de clientes.
 - Tratar la URL de la aplicación como información **confidencial** y no difundirla públicamente.
 - Para un acceso estrictamente privado, valorar el alojamiento en infraestructura propia con autenticación (por ejemplo, tras un proxy o VPN).
- **Rotación de credenciales:** ante cualquier exposición, revoque la credencial afectada, genere una nueva (Google Cloud / Supabase) y actualice los `Secrets`.

Anexo A · Carga de la base de datos en Supabase

El paquete incluye la base de datos ya preparada en el archivo `datos/db/agrapp_postgis.dump`, que contiene las tablas espaciales utilizadas por la aplicación:

Tabla	Contenido
<code>predios</code>	Polígonos catastrales (geometría <code>wkb_geometry</code>)
<code>construcciones_mvp</code>	Construcciones dentro de los predios
<code>frontera_mvp</code>	Frontera agrícola nacional

(La tabla de sistema `spatial_ref_sys` la crea automáticamente la extensión PostGIS al habilitarla, por lo que no requiere carga.)

El procedimiento consiste únicamente en **restaurar este archivo** en el proyecto Supabase del cliente.

A.1 · Requisitos

- Herramienta cliente `pg_restore` de PostgreSQL (versión 17 o superior), incluida en las «PostgreSQL client tools»:
 - **macOS:** `brew install libpq` y añada `libpq` al `PATH`.
 - **Windows:** instale «PostgreSQL» (incluye las client tools).
- Haber creado el proyecto Supabase **con región de EE. UU.** (Paso 4.1) y **habilitado la extensión `postgis`** (Paso 4.2) **antes** de restaurar.

A.2 · Restauración del archivo en Supabase

Desde la carpeta descomprimida, ejecute el siguiente comando (sustituyendo la contraseña y el proyecto):

```
pg_restore --no-owner --no-privileges \  
-d "postgresql://postgres:SU_PASSWORD@db.SU_PROYECTO.supabase.co:5432/postgres" \  
datos/db/agrapp_postgis.dump
```

Los avisos relativos a `postgis` o `spatial_ref_sys` (ya existentes al haber habilitado la extensión) son **normales y pueden ignorarse**.

A.3 · Verificación de la carga

En Supabase -> **SQL Editor**, ejecute:

```
select table_name from information_schema.tables  
where table_schema = 'public' order by 1;  
select count(*) from predios;  
select count(*) from construcciones_mvp;  
select count(*) from frontera_mvp;
```

Deben aparecer las 3 tablas, con un recuento de filas mayor que cero. A partir de este punto, la `DATABASE_URL` (Paso 6) apunta a una base con datos reales.

Soporte: para la carga de datos o cualquier duda de configuración, contacte con el equipo que le entregó la aplicación.